ORTA DOĞU TEKNİK ÜNİVERSİTESİ
MIDDLE EAST TECHNICAL UNIVERSITY
KUZEY KIBRIS KAMPUSU ◆ NORTHERN CYPRUS CAMPUS

CNG 495
Fall – 2024
Final Report

Name: Umutcan

Surname: CELIK

SID: 2526200

Project Title: ZeroDay Project

Date of Submission: 12.13.2024

Table of Contents

List of Figures

Project Description

The main goal of my project is to help new employees start their first day at work smoothly. With this project, companies can show new employees all the important documents on a website before they come to the office. They can also watch safety videos and other important videos online. This way, the first day will be easier, and both the new employee and the company will save time during the hiring process.
The project will be built using Python Flask, Heroku Postgres, Bootstrap, and Blueprint. Python Flask will be used to create the web interface and manage all the logic of the app. Heroku Postgres will store and manage the data in the cloud, making it easier to handle backups and scale up if more users join.
Bootstrap will be used to create a user-friendly interface that works well on all devices, like phones, tablets, and computers. Blueprint will help organize the code into different sections, making it easier to manage and reuse. Together, these tools will make the project more efficient, flexible, and easy to use for everyone.

Cloud Delivery Models

SaaS (Software as a Service):

I will utilize various SaaS solutions to enhance my project functionalities, providing user-friendly services for specific needs without going into specific providers.

PaaS (Platform as a Service):

I will use Heroku to deploy my application because it is a Platform as a Service (PaaS) that makes it easy to develop, run, and manage my Python Flask application. Heroku has many benefits, like automatic scaling, which adds more resources when my application gets more visitors, ensuring it runs smoothly. It also supports continuous integration and deployment (CI/CD), which helps me update my app quickly and efficiently.

IaaS (Infrastructure as a Service):

I will use Heroku Postgres as my database solution. Heroku Postgres provides scalable database services that can easily integrate with my application, allowing me to manage and store data efficiently in the cloud.

Diagrams

Use Case Diagram

You can see the use case diagram at Figure 1.

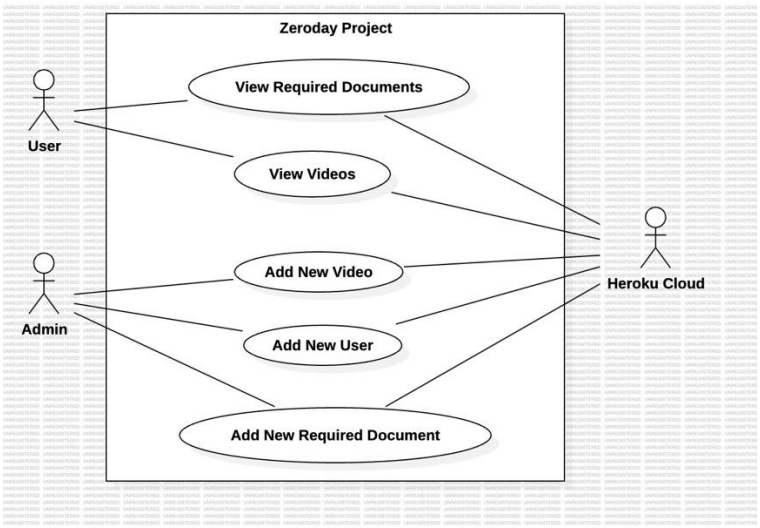


Figure 1: use case diagram of project

Data Flow Diagram

You can see the data flow diagram at Figure 2 and Figure 3.

Context Level:

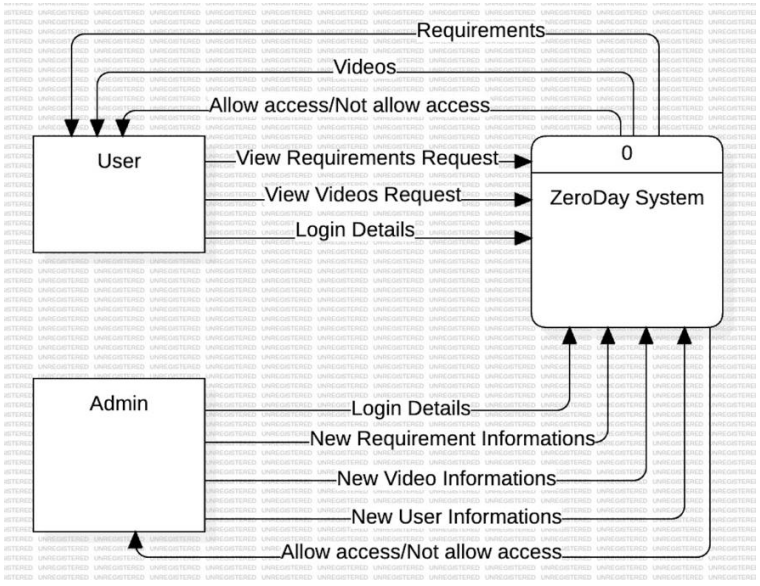


Figure 2: context level of project

Level 0:

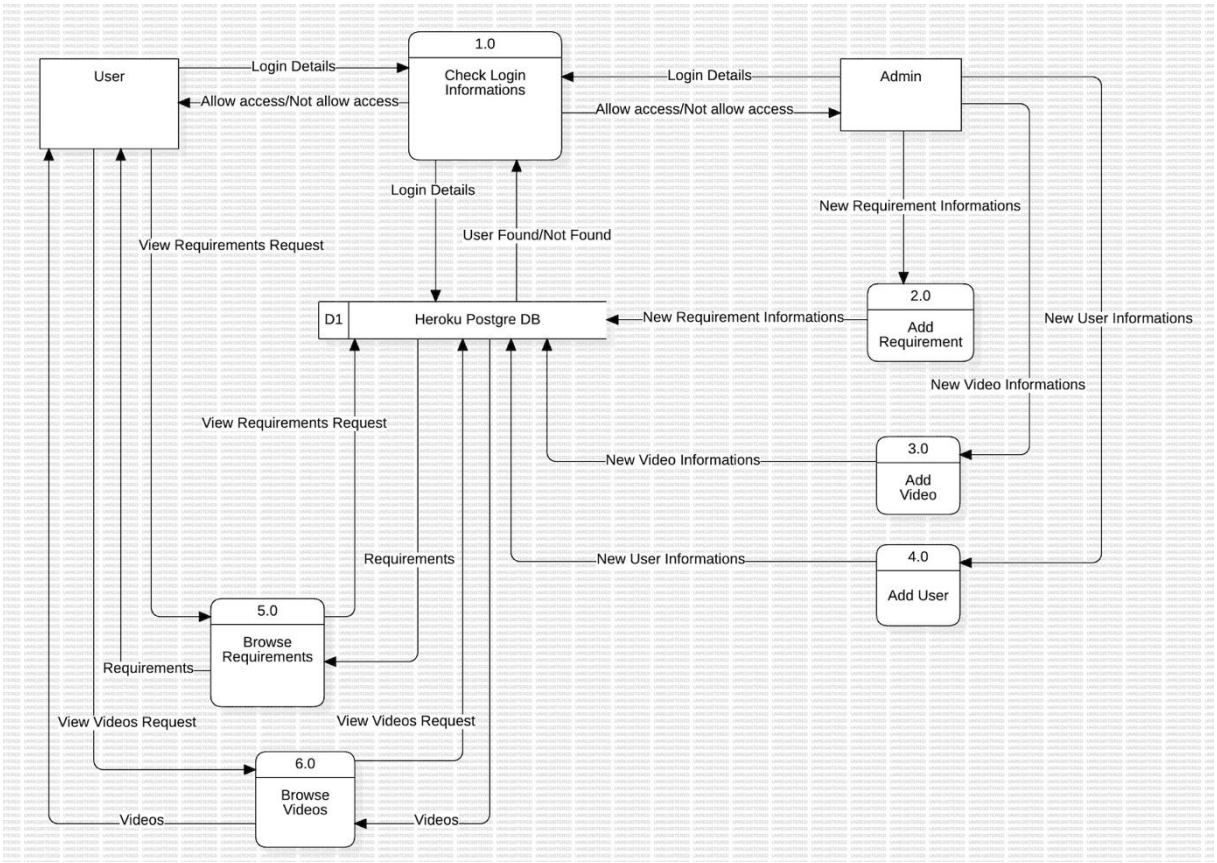


Figure 3: Level 0 of project

Data Types

In my project, the types of data I use are:

Text Data:

- Video Names and Links: In the "videos" section, I show the names and links of the videos that new employees can watch before their first day at work.
- User Login Information: Users log in to the system using a username and a password to access the platform.
- Data Stored in the Database: The database in my project stores important information such as usernames, passwords, video names, links, like/dislike counts, and required documents in text format.

Computation

In my project, computation involves processing data and performing calculations. For example:

- Checking user login information when users access the platform.
- Updating the like/dislike counts for each video when users interact with them.
- Adding new user (create username, password and decide he/she is admin or not).

Expected Contribution

The backend, frontend, database and cloud parts of the project will be done by Umutcan Celik.

Releated Work

1. SAP SuccessFactors Onboarding: This solution simplifies the onboarding process for new employees by integrating systems, processes, and people into a mobile-friendly and intuitive experience.[6]
2. Workday Onboarding: Workday's onboarding module helps new employees quickly adapt to company culture and complete necessary documentation, streamlining the hiring process. [7]
3. BambooHR Onboarding: Designed for small and medium-sized businesses, BambooHR offers a user-friendly onboarding process to help new employees prepare for their first day. [8]
4. Google Workspace Onboarding: Google Workspace facilitates access to company resources and collaboration for new employees, ensuring a smooth onboarding process. [9]

Novelties of the Project

- Pre-Onboarding Focus: Unlike traditional systems, the project prepares employees before their first day by providing access to key documents and videos online, saving time during onboarding.
- Cloud Accessibility: Hosting on Heroku and using Heroku Postgres ensures scalability, reliability, and easy access from anywhere.

- Dynamic Content Management: Admins can easily update requirements and videos, making the system adaptable to company-specific needs.

- Modern Tech Stack: Combines Python Flask, Bootstrap, and Flask Blueprints for a scalable, responsive, and modular design.

Contribution to the Field

The project modernizes onboarding by focusing on pre-arrival preparation, offering a scalable and user-friendly platform that saves time and enhances the onboarding process for both companies and employees.

Milestones Achieved

Week 4 (Oct 21 - 27) -- Week 5 (Oct 28 - Nov 3)

I completed my HTML and CSS codes. In this section, I added comments to keep the codes organized and readable and separated my CSS and HTML codes under static/css and templates files. You can see from Figure 4.

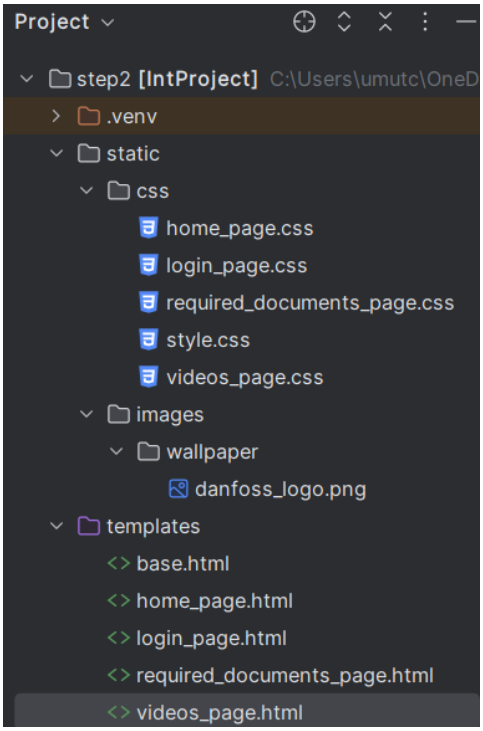


Figure 4: HTML and CSS code layout

You can see the interfaces created by the HTML and CSS codes I wrote from figure 5,6,7,8.

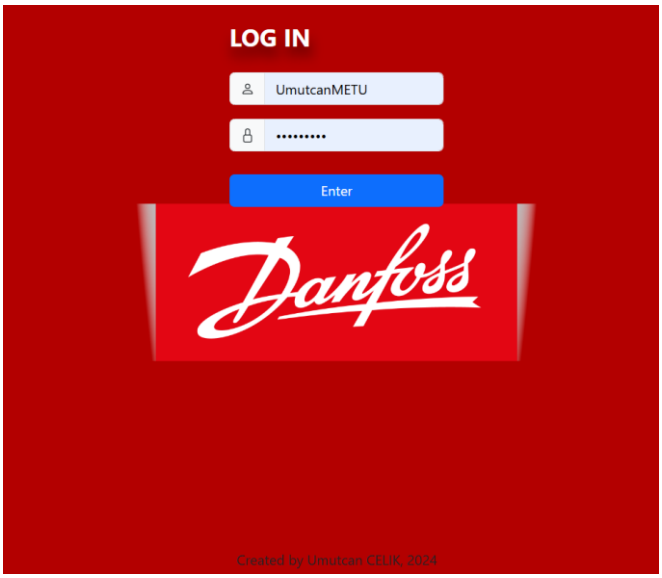


Figure 5: login screen

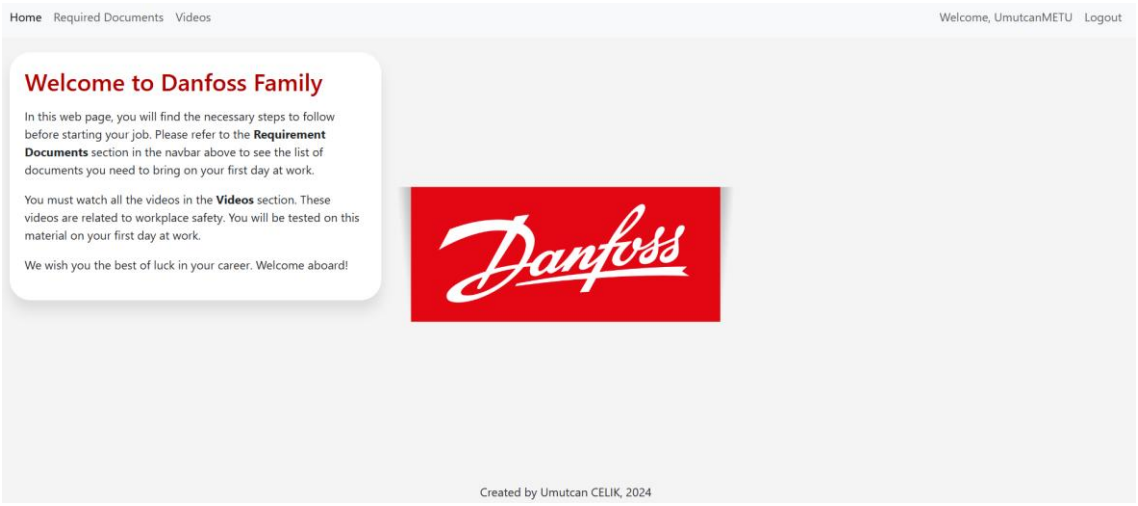


Figure 6: home page screen

Figure 7: documents screen

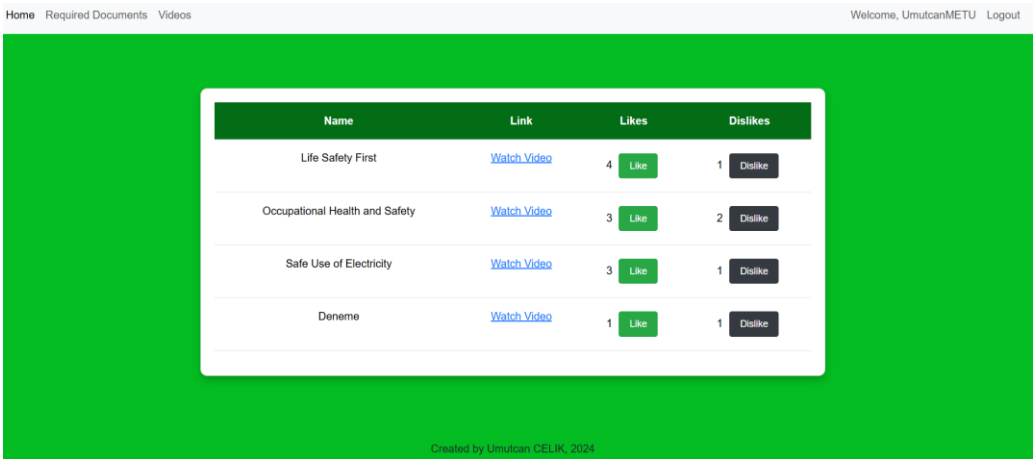


Figure 8: videos screen

Week 6 (Nov 4 - 10)

In these weeks, I designed the DB that I will use in PostgreSQL and added a small number of data in them. You can check my work on this from figure 9,10,11.

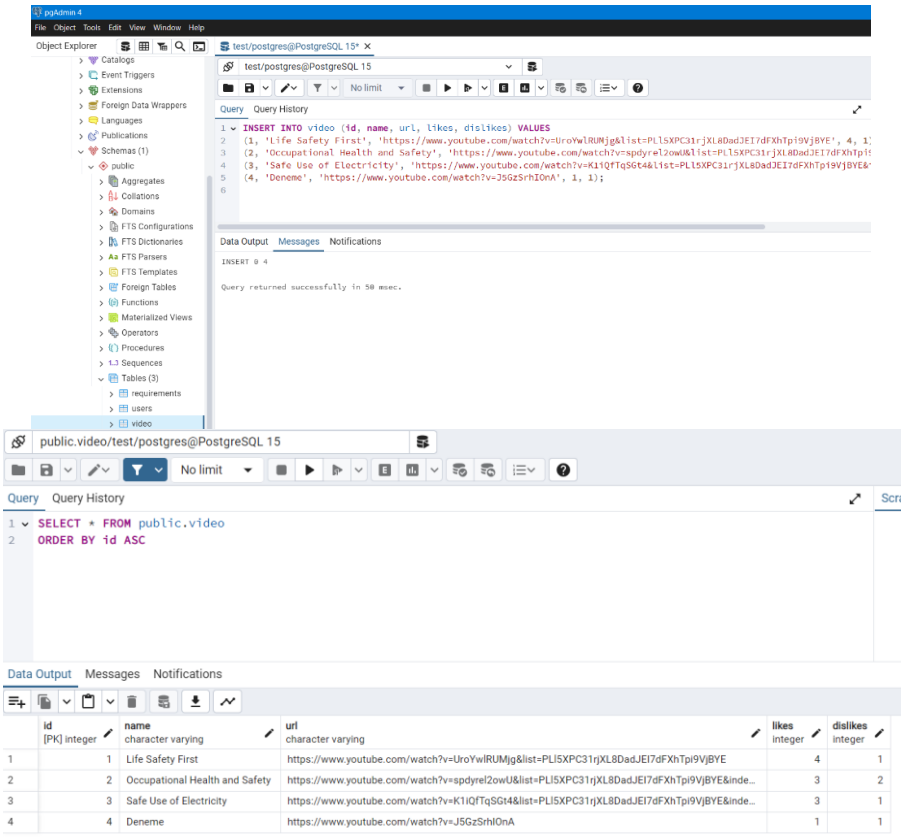


Figure 9: create DB-1

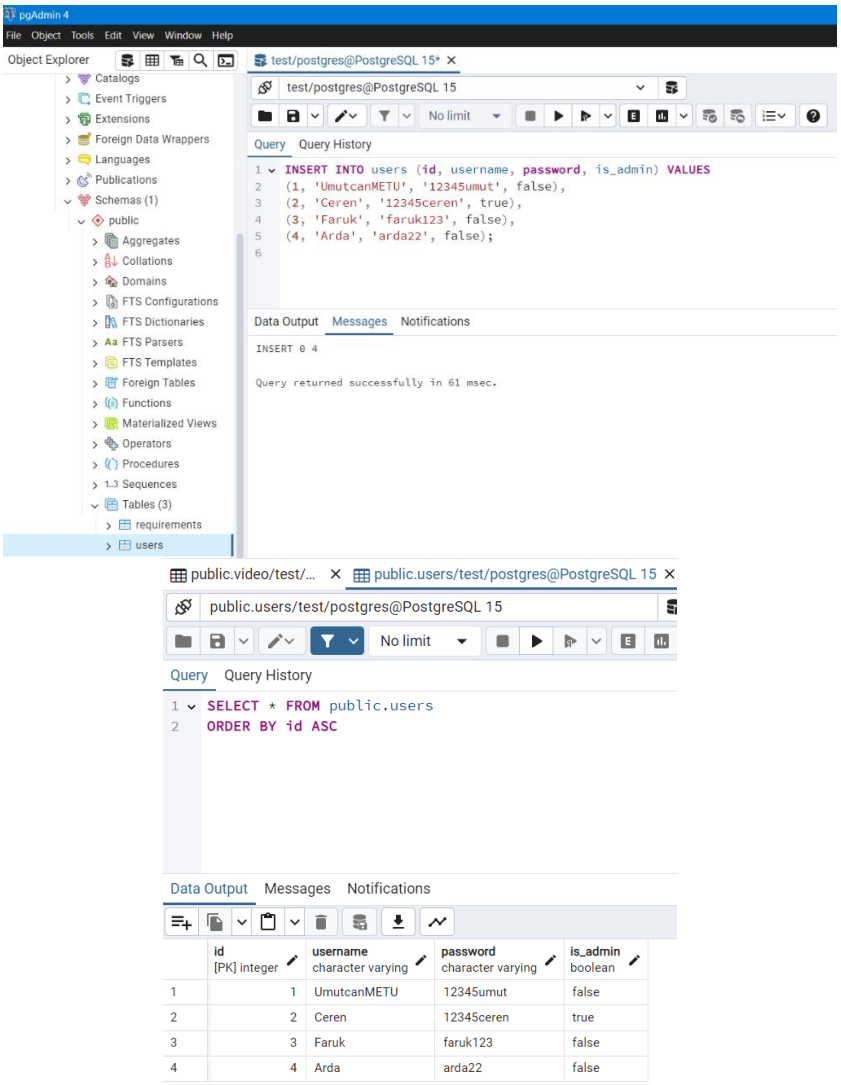


Figure 10: create DB-2

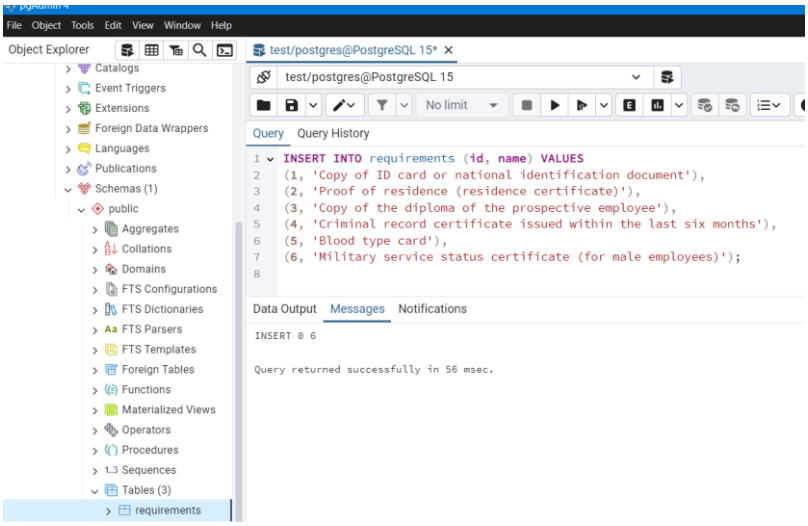


Figure 11: create DB-3

Week 7 (Nov 11 - 17) -- Week 8 (Nov 18 - 24)

Between these weeks I completed the main part of my code. You can see the file structure of my main code in figure 12. Since I am using Blueprint, I have added these codes by opening an extra file named views.

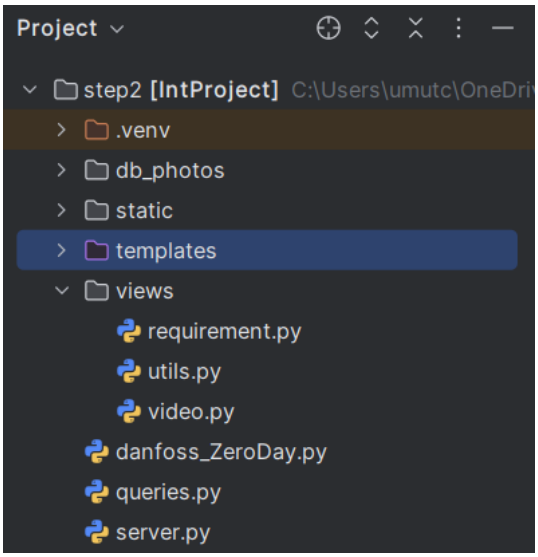


Figure 12: main codes structure

I want to talk a little bit about the important parts of my code. You can see my server.py code in Figure 13.

server.py

```python
from flask import Flask, redirect, url_for, render_template, request, session, flash
from psycopg2 import extensions
from queries import *

# Importing the requirement Blueprint
from danfoss_ZeroDay import initialize
from views.requirement import requirement
from views.utils import login_required
from views.video import video

# Register Unicode extensions for PostgreSQL
extensions.register_type(extensions.UNICODE)
extensions.register_type(extensions.UNICODEARRAY)

</
app = Flask(__name__)
app.secret_key = 'METUNCC'  # Secret key for session management

# Registering Blueprints for modular route handling
# Registering Blueprints for modular route handling
</requirement
app.register_blueprint(requirement, url_prefix="/requirement")
</videos
app.register_blueprint(video, url_prefix="/videos")

# Database configuration
HEROKU = False
if not HEROKU:
    os.environ['DATABASE_URL'] = "dbname='test' user='postgres' host='localhost' password='umut62'"
    initialize(os.environ.get('DATABASE_URL'))  # Initialize the database connection
```

```python
# Route for the home page, requires user to be logged in
@app.route("/")  3 usages
@login_required  # Ensure the user is logged in
def home_page():
    return render_template("home_page.html")  # Render the home page template

# Route for the login page, handles GET and POST methods
/login
@app.route( rule: "/login", methods=["GET", "POST"])  6 usages
def login():
    if 'username' in session:  # Check if the user is logged in
        return redirect(url_for('home_page'))  # Redirect to the home page

    if request.method == "POST":  # If the form is submitted
        username = request.form.get("username")  # Get username
        password = request.form.get("password")  # Get password

        # Validate the input fields
        if not username:
            flash( message: "Username cannot be empty.", category: "warning")
            return redirect(url_for('login'))  # Redirect back to log in
        if not password:
            flash( message: "Password cannot be empty.", category: "warning")
            return redirect(url_for('login'))  # Redirect back to log in

        # Query to get user information from the database
        user = select( columns: "username, password, is_admin", table: "users", where: f"username='{username}'", asDict=True)

        # Check if the user exists and validate the password
        if not user:
                flash( message: "No such username found.", category: "danger")
                return redirect(url_for('login'))  # Redirect back to log in
            elif user['password'] != password:  # If the password is incorrect
                flash( message: "Incorrect password.", category: "danger")
                return redirect(url_for('login'))  # Redirect back to log in
            else:
                # Set session variables for the logged-in user
                session['username'] = user['username']
                session['is_admin'] = user['is_admin']  # Add the admin status to the session
                return redirect(url_for('home_page'))  # Redirect to the home page

        return render_template("login_page.html")  # Render the login page template

# Route for logging out, requires user to be logged in
/logout
@app.route("/logout")  1 usage
@login_required
def logout():
    session.pop('username', None)  # Remove the username from the session
    return redirect(url_for('login'))

# Main entry point for the application
if __name__ == "__main__":
    app.run(debug=True if not HEROKU else False)  # Run the app in debug mode for local development
```

Figure 13: server.py

First, I imported necessary libraries for routing, templates, and database functions. Then, I set up **Blueprints** to keep things organized, like `/requirement` and `/videos` sections, which helps to manage routes separately. I also configured the PostgreSQL database connection for local development and prepared it for deployment on Heroku. The main parts include a **login system** where users enter a username and password, and I check if the info matches the database. If correct, the user can access the home page; if not, they get a warning. I also added a **logout** function to clear the session. Finally, the app runs in debug mode locally but will be turned off on Heroku. Next steps could involve adding admin features for more control over content.

Now I will talk about my danfoss_ZeroDay.py code where I manage my DB. In my code, I set up the database structure for my app using **PostgreSQL**. First, I imported libraries to work with environment variables and database connections. Then, I created some **SQL statements** that make important tables if they don't already exist. These include a user's table for storing usernames and passwords, a video table for videos with likes and dislikes, and a requirements table for important items. I also added an is_admin column to the users table to identify admin users. The initialize function runs these SQL statements when I connect to the database, setting up the tables automatically. I included a get_all_requirements function to get all entries from

the requirements table. Finally, I added a part at the end to make sure the database URL is set; if not, it gives a message and stops. Next steps might include adding more functions to manage data inside these tables more easily. You can examine my code from Figure 14 and 15.

danfoss_ZeroDay.py

```python
import os  # For accessing environment variables
import sys  # For system-specific parameters and functions
import psycopg2 as dbapi2  # PostgreSQL database interaction

# SQL statements to initialize the database tables
INIT_STATEMENTS = [
    """
    create table if not exists users(
        id serial primary key,
        username varchar not null unique,
        password varchar not null
    )
    """,
    """
    create table if not exists video(
        id serial primary key,
        name varchar not null,
        url varchar not null,
        likes integer default 0 not null,
        dislikes integer default 0 not null
    )
    """,
    """
    ALTER TABLE users ADD COLUMN IF NOT EXISTS is_admin BOOLEAN DEFAULT FALSE
    """,
    """
    create table if not exists requirements(
        id serial primary key,
        name varchar not null
    )
    """
]
```

Figure 14: danfoss_ZeroDay.py1

```python
# Initialize the database with the given URL
def initialize(url):  3 usages
    with dbapi2.connect(url) as connection:
        cursor = connection.cursor()
        for statement in INIT_STATEMENTS:
            cursor.execute(statement)
        cursor.close()

# Function to get all requirements
def get_all_requirements():  2 usages
    """Retrieve all requirements from the requirements table."""
    query = "SELECT id, name FROM requirements"
    with dbapi2.connect(os.getenv("DATABASE_URL")) as connection:
        cursor = connection.cursor()
        cursor.execute(query)
        requirements = cursor.fetchall()
    return requirements

# Main execution to run the initialization
if __name__ == "__main__":
    url = os.getenv("DATABASE_URL")
    if url is None:
        print("Usage: DATABASE_URL=url python danfoss_ZeroDay.py")
        sys.exit(1)
    initialize(url)
```

Figure 15: danfoss_ZeroDay.py2

While writing these codes, of course I also did tests, you can find these tests in figure 16.

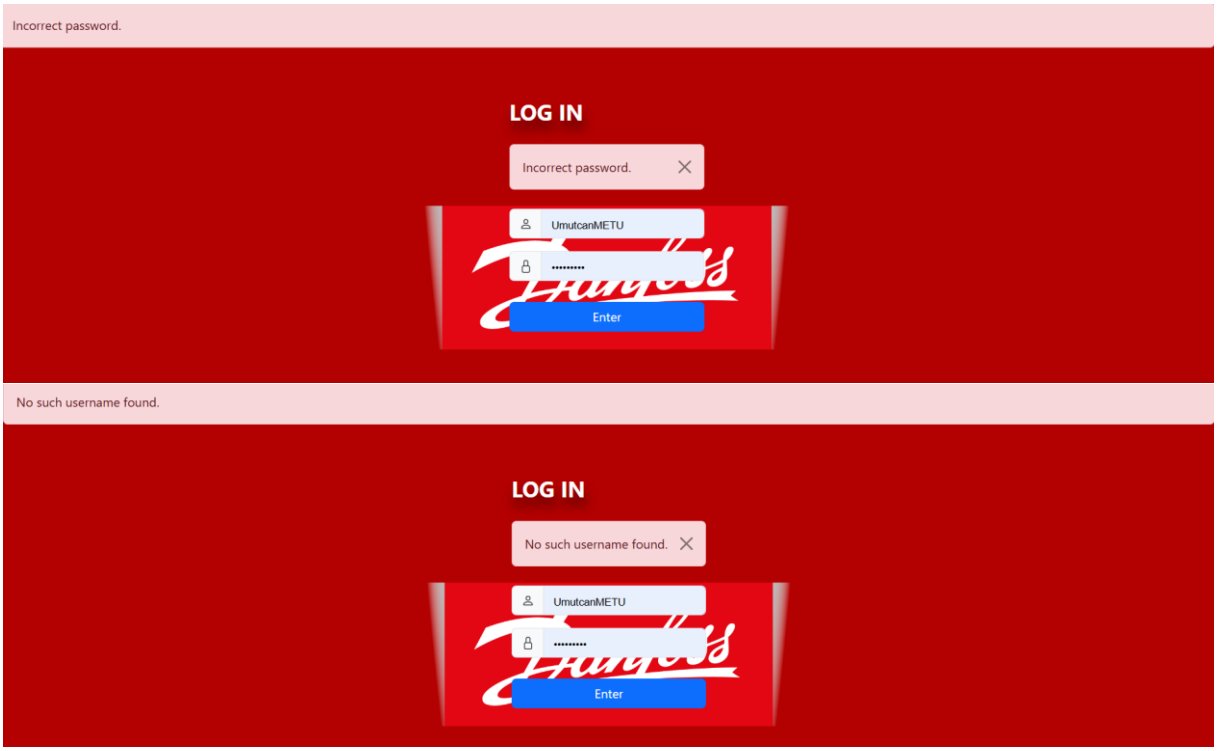


Figure 16: test

The test I did in Figure 17 was to check if the DB was updated for the videos.

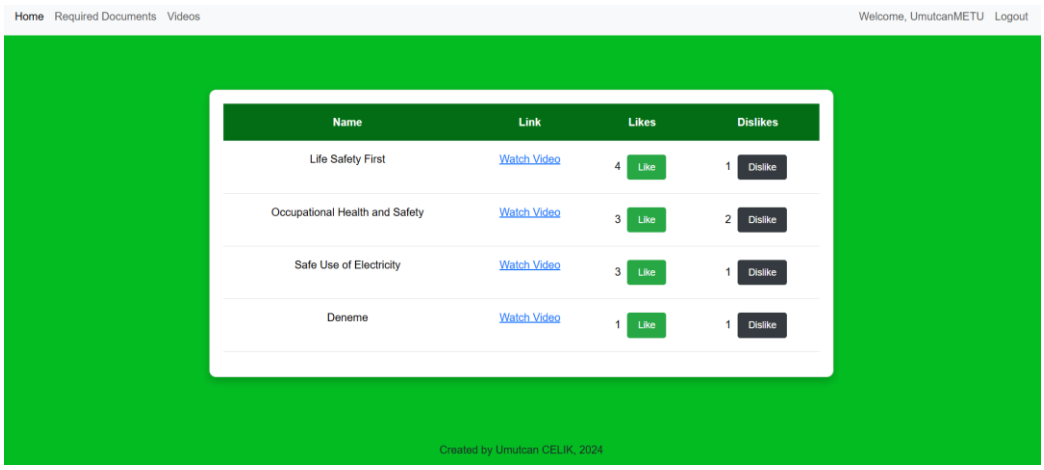


Figure 17: test DB

After creating a Heroku account, I uploaded a test code and learned how to integrate it with postgresql. You can see this process in figure 18.

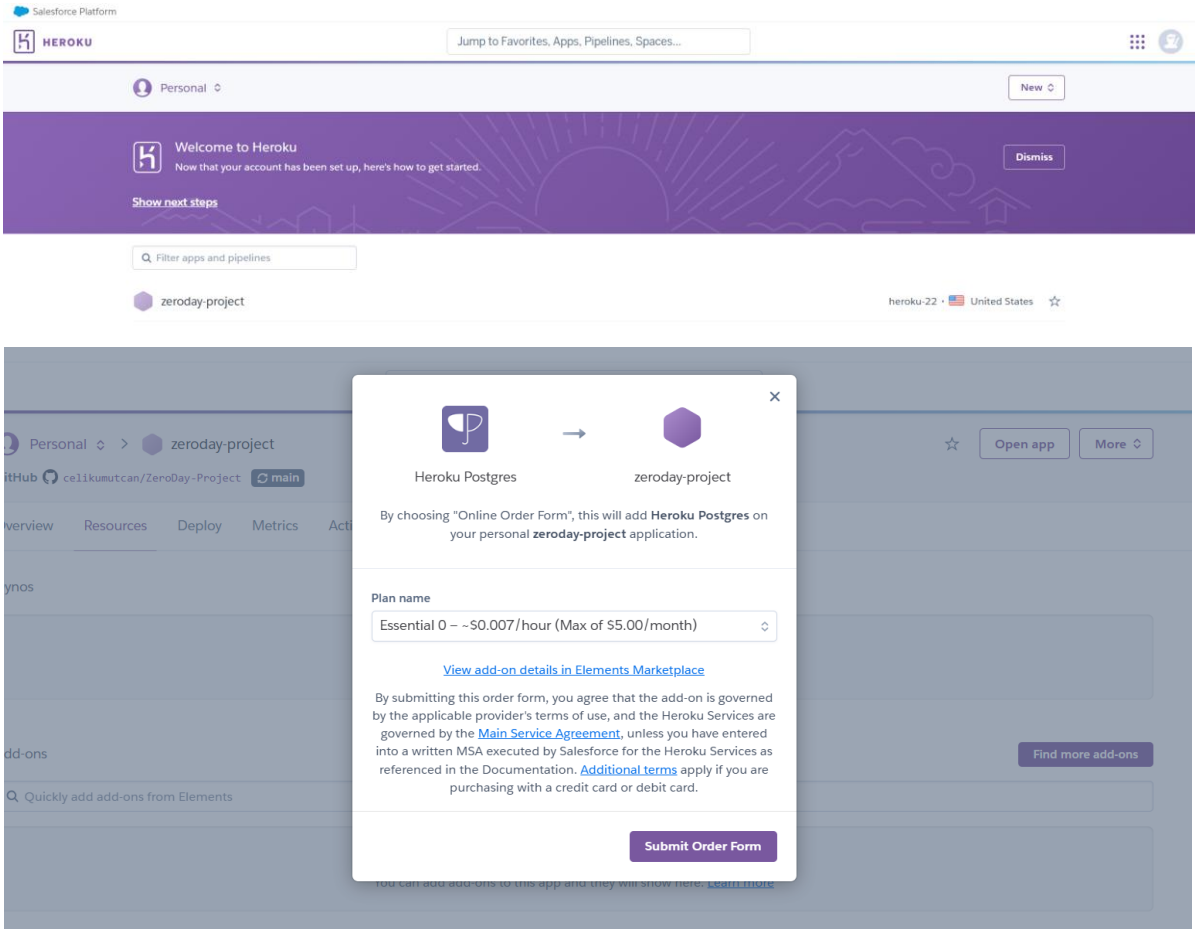
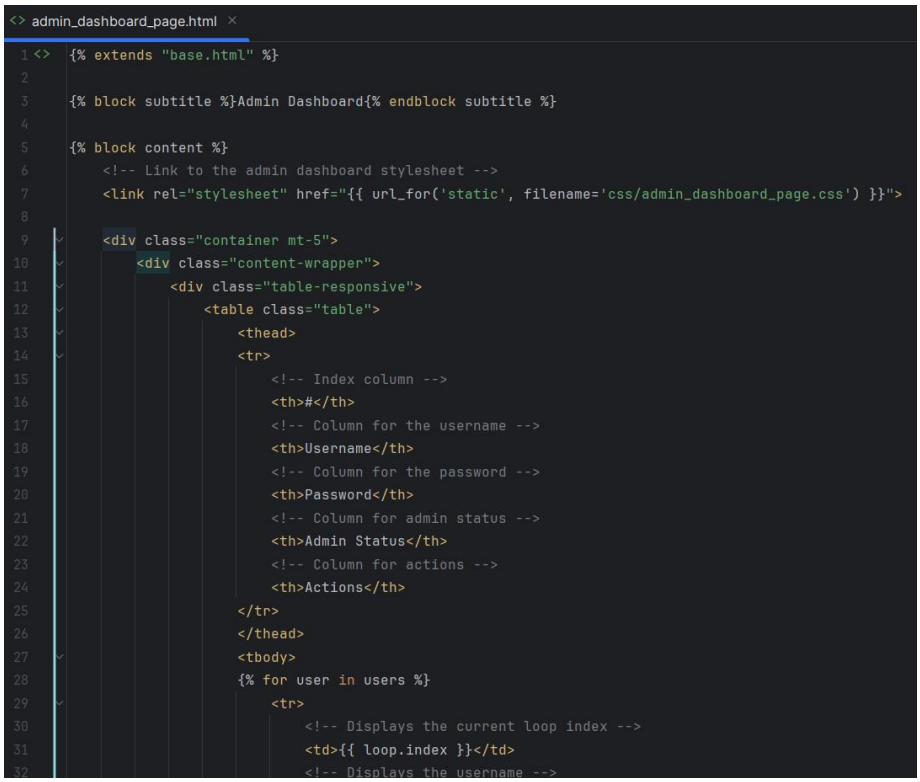


Figure 18: create Heroku account

The difficulties I had in my codes that I have completed so far were especially difficult to organize and connect the code accordingly, especially since I learned to use Blueprint from the beginning. Even though I didn't take a course on Pyton-Flask, creating this code took me time to research and learn. I made mistakes, but in the end I got a good result. For Heroku, I recommend using the recommended "Salesforce Authenticator" to avoid the "Authenticator" problem. I used "Microsoft Authenticator" and after a while I couldn't access the account. I contacted Help Support and after 2 days I was able to reactivate the account.

Week 9 (Nov 25 - Dec 1)

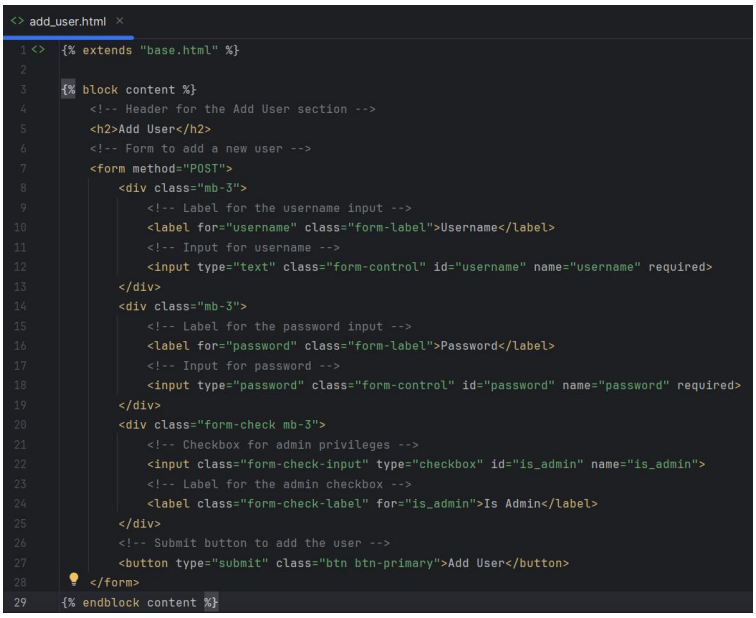
First, I created the HTML codes for the admin-specific interfaces. This code in figure 19, creates an admin dashboard page where administrators can view, add, and delete users with a responsive interface.



```
<> admin_dashboard_page.html ×
1 <> {% extends "base.html" %}
2
3   {% block subtitle %}Admin Dashboard{% endblock subtitle %}
4
5   {% block content %}
6       <!-- Link to the admin dashboard stylesheet -->
7       <link rel="stylesheet" href="{{ url_for('static', filename='css/admin_dashboard_page.css') }}">
8
9       <div class="container mt-5">
10          <div class="content-wrapper">
11              <div class="table-responsive">
12                  <table class="table">
13                      <thead>
14                      <tr>
15                          <!-- Index column -->
16                          <th>#</th>
17                          <!-- Column for the username -->
18                          <th>Username</th>
19                          <!-- Column for the password -->
20                          <th>Password</th>
21                          <!-- Column for admin status -->
22                          <th>Admin Status</th>
23                          <!-- Column for actions -->
24                          <th>Actions</th>
25                      </tr>
26                      </thead>
27                      <tbody>
28                      {% for user in users %}
29                          <tr>
30                              <!-- Displays the current loop index -->
31                              <td>{{ loop.index }}</td>
32                              <!-- Displays the username -->
```

Figure 19: admin_dashboard_page

Next, I created the HTML code for adding new users. This code in figure 20 allows administrators to add a user by providing a username, password, and selecting whether the user has admin privileges through a simple form.



```
<> add_user.html ×
1 <> {% extends "base.html" %}
2
3   {% block content %}
4       <!-- Header for the Add User section -->
5       <h2>Add User</h2>
6       <!-- Form to add a new user -->
7       <form method="POST">
8           <div class="mb-3">
9               <!-- Label for the username input -->
10              <label for="username" class="form-label">Username</label>
11              <!-- Input for username -->
12              <input type="text" class="form-control" id="username" name="username" required>
13          </div>
14          <div class="mb-3">
15              <!-- Label for the password input -->
16              <label for="password" class="form-label">Password</label>
17              <!-- Input for password -->
18              <input type="password" class="form-control" id="password" name="password" required>
19          </div>
20          <div class="form-check mb-3">
21              <!-- Checkbox for admin privileges -->
22              <input class="form-check-input" type="checkbox" id="is_admin" name="is_admin">
23              <!-- Label for the admin checkbox -->
24              <label class="form-check-label" for="is_admin">Is Admin</label>
25          </div>
26          <!-- Submit button to add the user -->
27          <button type="submit" class="btn btn-primary">Add User</button>
28      </form>
29  {% endblock content %}
```

Figure 20: add_user

After that, I designed the CSS for the admin dashboard page. This code in figure 21 customizes the page by setting a golden background, styling the container with rounded corners and shadows, and enhancing the table with a polished look, including red borders and bold headers for a clean and professional interface.

admin_dashboard_page.css

```css
/* Sets the background color and font for the entire page */
body {
    background-color: #cc8f00;
    font-family: 'Arial', sans-serif;
}

/* Styles the container with background color, border radius, shadow, padding, and top margin */
.container {
    background-color: rgb(202, 202, 202);
    border-radius: 15px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
    padding: 30px;
    margin-top: 50px;
}

/* Styles the table with border, border radius, overflow, and shadow */
.table {
    border-collapse: separate;
    border-spacing: 0;
    border: 1px solid #df0707;
    border-radius: 10px;
    overflow: hidden;
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
}

/* Adds padding and bottom border to table cells, aligns text to the left */
.table th,
.table td {
    padding: 15px;
    text-align: left;
    border-bottom: 1px solid #ddd;
}
```

Figure 21: admin_dashboard_page_css

Then, I implemented the backend functionality for the admin panel using Flask Blueprints, enabling administrators to manage users through routes that display the user dashboard, handle the addition of new users via a form, and allow user deletion by ID, with feedback provided through flash messages. You can check from figure 22.

admin.py

```python
import ...

# Create the admin Blueprint with a specific URL prefix
</admin
admin = Blueprint( name: 'admin', __name__, template_folder='templates')  # Prefix can be added during registration

# Renders the admin dashboard page with a list of users
</admin/dashboard
@admin.route('/dashboard')  1 UMO
def dashboard():
    users = get_all_users()  # Fetches all users from the database
    return render_template( template_name_or_list: 'admin/admin_dashboard_page.html', users=users)

# Handles adding a new user, accessible via GET and POST methods
</admin/add_user
@admin.route( rule: '/add_user', methods=['GET', 'POST'])  1 UMO
def add_user_route():
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')
        is_admin = request.form.get('is_admin') == 'on'

        if username and password:
            add_user(username, password, is_admin)
            flash( message: "User added successfully!",  category: "success")
            return redirect(url_for('admin.dashboard'))
        else:
            flash( message: "Please provide a valid username and password.",  category: "danger")
    return render_template('admin/add_user.html')

# Deletes a user based on the provided user ID
</admin/delete_user/<user_id>
```

Figure 22: admin.py

To enhance the admin features, I updated the database functionality by adding operations for managing users and requirements. Functions such as add_user, get_all_users, and delete_user_by_id were implemented to allow adding, viewing, and deleting users, with support for setting admin status. Additionally, I added add_requirement_to_db, get_all_requirements, and delete_requirement_by_id to provide admins the ability to dynamically manage the requirements list. These updates ensure that the admin panel is fully functional and integrated with the backend. You can check from figure 23.

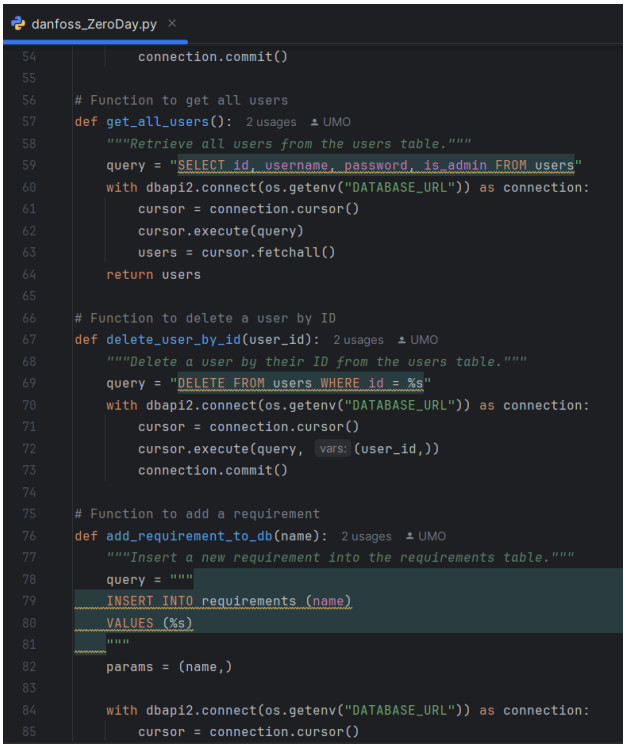


Figure 23: danfoss_ZeroDay.py3

Finally, I updated the server.py file to integrate the admin features into the main application. I registered the admin Blueprint with the URL prefix /admin to modularly handle all admin-related routes. Additionally, I ensured that the database initialization (initialize) includes the required tables for managing admin functionalities. These adjustments allow seamless integration of the admin dashboard into the project, providing administrators with tools to manage users and requirements effectively. You can see from the figure below.

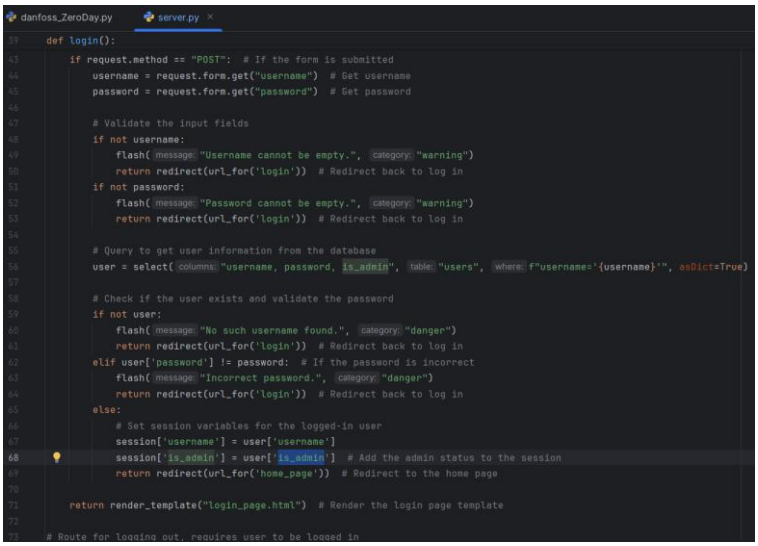


Figure 24: server.py2

Week 10 (Dec 2 - 8)

That week, I expanded my system by adding 146 more users. You can see the SQL code for this in Figure 25. This allowed me to populate the users table with a total of 150 entries, including one additional admin user, "METU" By doing so, I ensured a more realistic database for testing the admin dashboard and user management functionalities. [5]

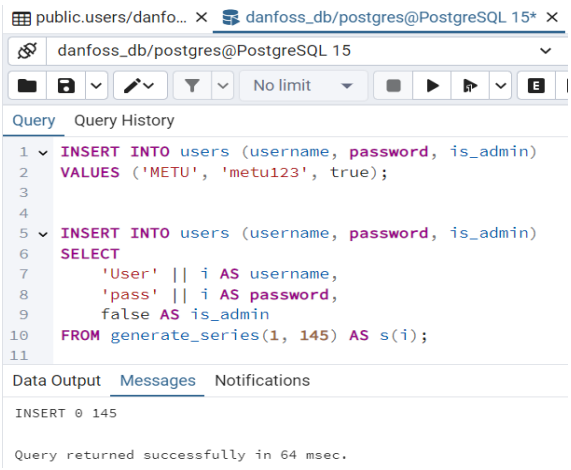


Figure 25: multiple addition

You can see from Figure 26 that the data was recorded successfully.

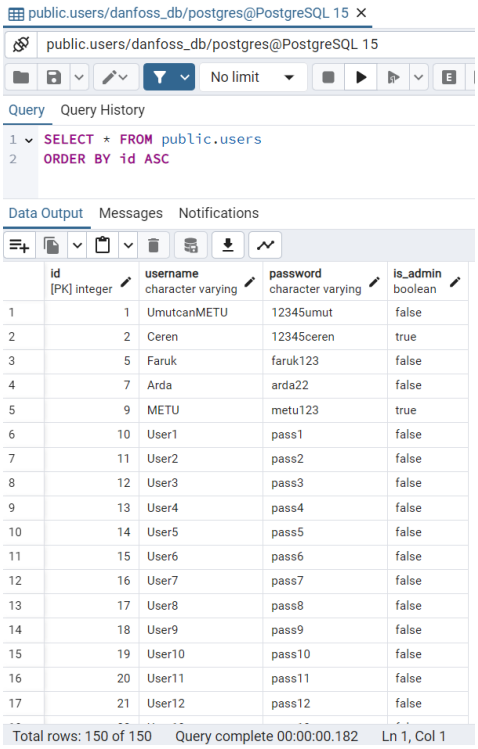


Figure 26: users

If you want, let's look at the images of my updated project now. You can examine the homepage that opens when the admin logs into the system in Figure 27.

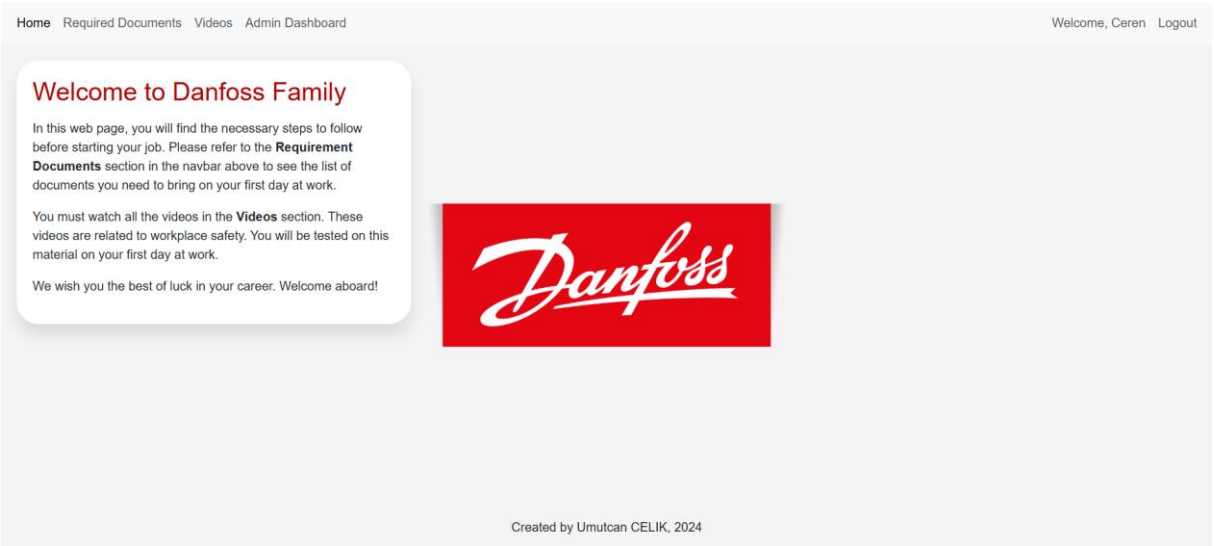


Figure 27: admin_homepage

You can see from the Required Documents section in Figure 28 that the admin can add new requirements and delete old ones.

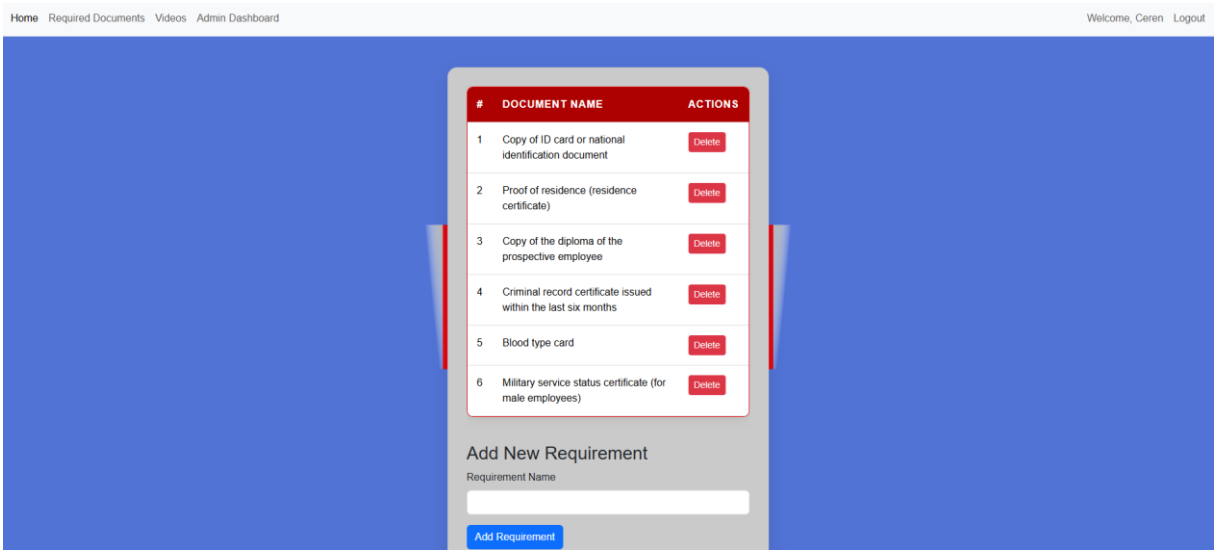


Figure 28: admin_requiredDocuments

From Figure 29 you can see that the admin can add and delete videos to the video page.

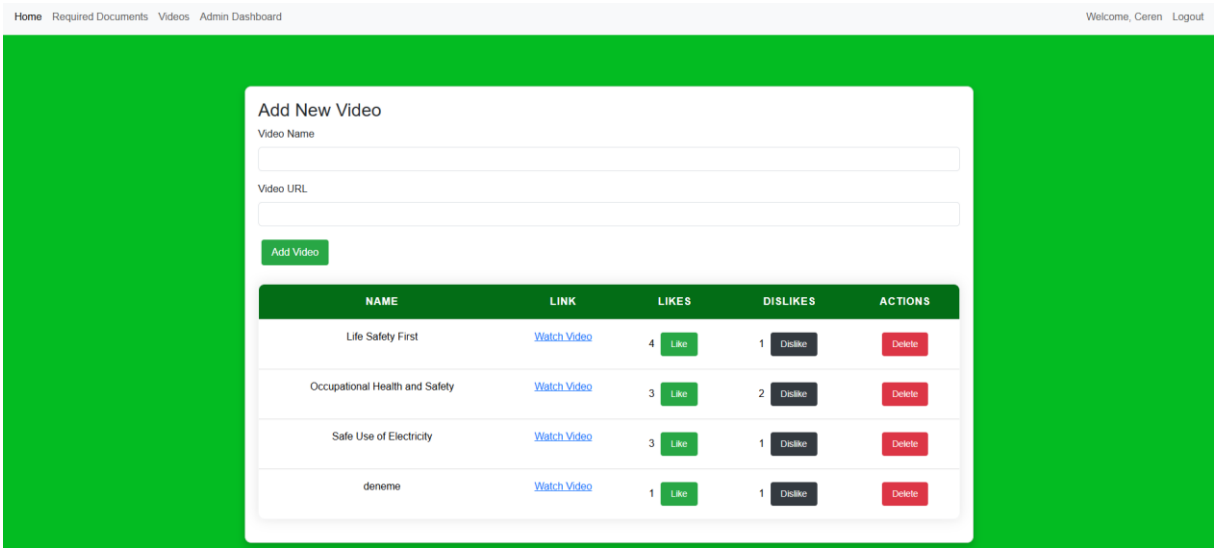


Figure 29: admin_videopage

And finally, you can see (from figure 30) that the admin can easily change users via the system without entering the DB. From here, they can add a new user and give permission to use them by giving them their username and password via email.

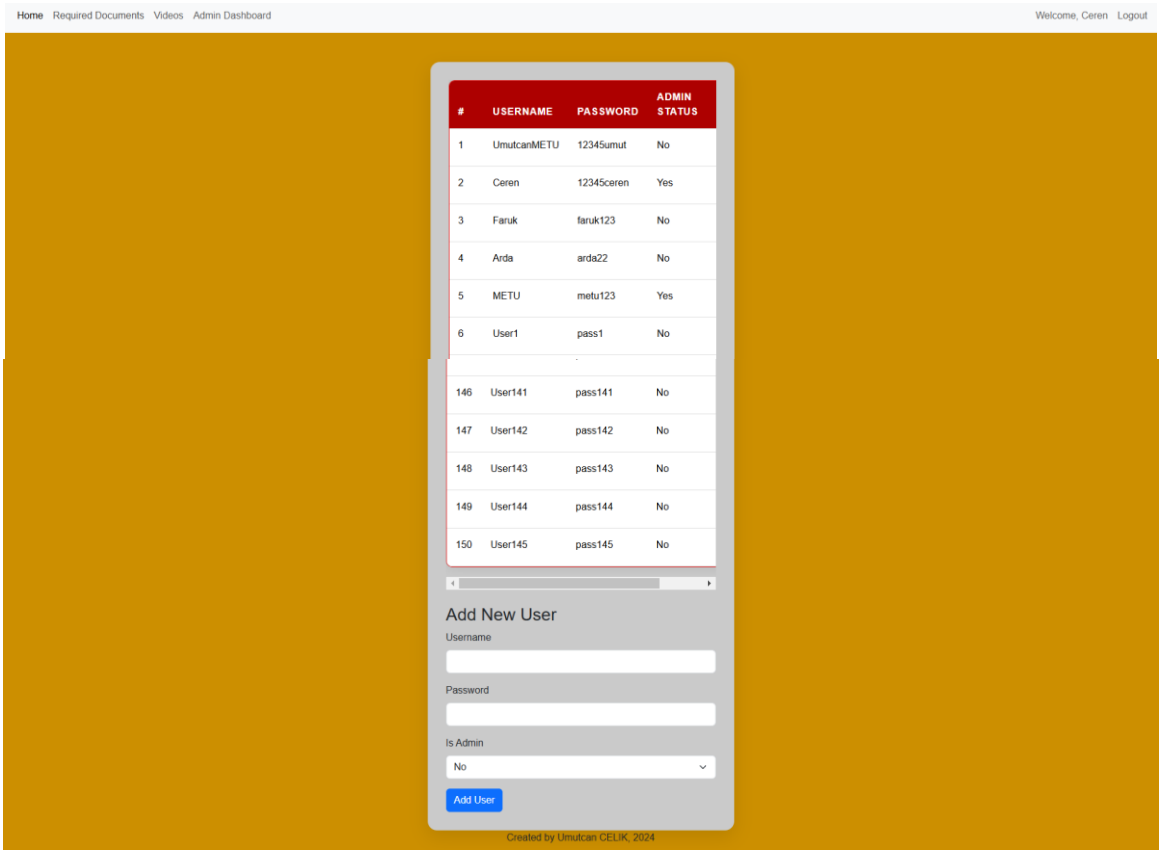


Figure 30: admin_dashboardPage

Week 11 (Dec 9 – Dec 15)

First of all, I connected the codes I uploaded to GitHub with my Heroku account. You can see this in figure 31.

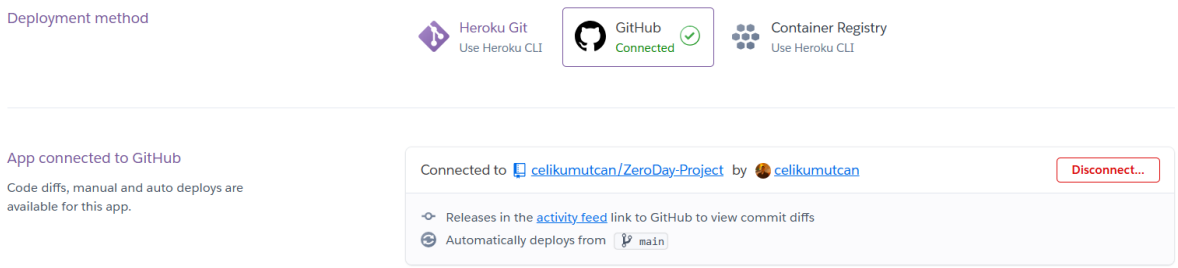


Figure 31: connect Heroku with GitHub

If you recall, we completed the connection with Heroku Postgres in figure 18. I am pulling our codes from GitHub to Heroku with Deploy Branch. You can see this in figure 32.

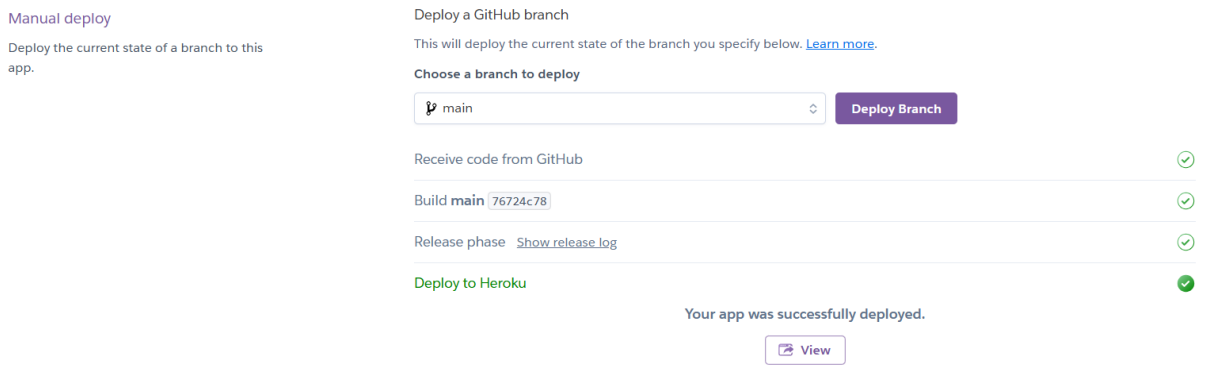


Figure 32: connect codes from GitHub

We move our tables to Heroku with the help of the code in Figure 33.

```
Microsoft Windows [Version 10.0.26100.2454]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\umutc\OneDrive - metu.edu.tr\Desktop\Final_Version>heroku psql -a zeroday-project
--> Connecting to postgresql-round-23962
psql (15.8, server 16.4)
WARNING: psql major version 15, server major version 16.
         Some psql features might not work.
WARNING: Console code page (857) differs from Windows code page (1254)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

zeroday-project::DATABASE=> \dt;
              List of relations
 Schema |     Name     | Type  |     Owner
--------+--------------+-------+---------------
 public | requirements | table | u8cm6dcsf2a3u5
 public | users        | table | u8cm6dcsf2a3u5
 public | video        | table | u8cm6dcsf2a3u5
(3 rows)


zeroday-project::DATABASE=>
```

Figure 33: connect tables

Then I added my insert codes. You can see it in Figure 34.

```
zeroday-project::DATABASE=> INSERT INTO users (username, password, is_admin)
zeroday-project::DATABASE-> SELECT
zeroday-project::DATABASE->     'User' || i AS username,
zeroday-project::DATABASE->     'pass' || i AS password,
zeroday-project::DATABASE->     false AS is_admin
zeroday-project::DATABASE-> FROM generate_series(1, 148) AS s(i);
INSERT 0 148
zeroday-project::DATABASE=> INSERT INTO requirements (id, name) VALUES
zeroday-project::DATABASE-> (1, 'Copy of ID card or national identification document'),
zeroday-project::DATABASE-> (2, 'Proof of residence (residence certificate)'),
zeroday-project::DATABASE-> (3, 'Copy of the diploma of the prospective employee'),
zeroday-project::DATABASE-> (4, 'Criminal record certificate issued within the last six months'),
zeroday-project::DATABASE-> (5, 'Blood type card'),
zeroday-project::DATABASE-> (6, 'Military service status certificate (for male employees)');
INSERT 0 6
zeroday-project::DATABASE=> INSERT INTO video (id, name, url, likes, dislikes) VALUES
zeroday-project::DATABASE-> (1, 'Life Safety First', 'https://www.youtube.com/watch?v=UroYwIRUMjg&list=PLI5XPC31rjXL8DadJEI7dFXhTpi9VjBYE', 4, 1),
zeroday-project::DATABASE-> (2, 'Occupational Health and Safety', 'https://www.youtube.com/watch?v=spdyrel2owU&list=PLI5XPC31rjXL8DadJEI7dFXhTpi9VjBYE&index=3', 3, 2),
zeroday-project::DATABASE-> (3, 'Safe Use of Electricity', 'https://www.youtube.com/watch?v=K1iQfTqSGt4&list=PLI5XPC31rjXL8DadJEI7dFXhTpi9VjBYE&index=8', 3, 3),
zeroday-project::DATABASE-> (4, 'deneme', 'https://www.youtube.com/watch?v=J5GzSrhlOnA', 1, 1);
INSERT 0 4
zeroday-project::DATABASE=>
```

Figure 33: insert data at tables

After adding the project to Heroku, I noticed that the requirements table was not updated. It was giving an error message. You can see this error in Figure 34.

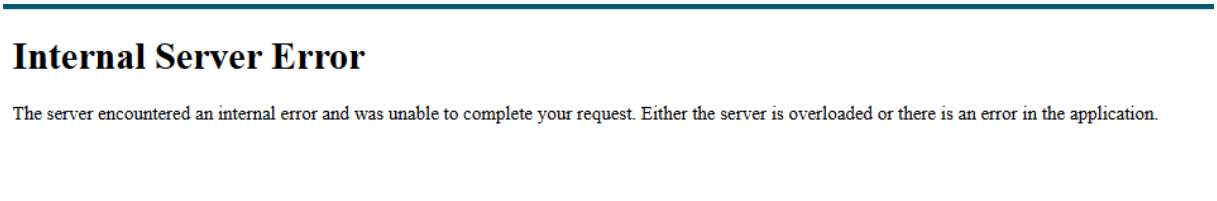


Figure 34: error

While trying to solve this problem, I checked if there were any errors with the code I used in Figure 35. I couldn't see any problems.

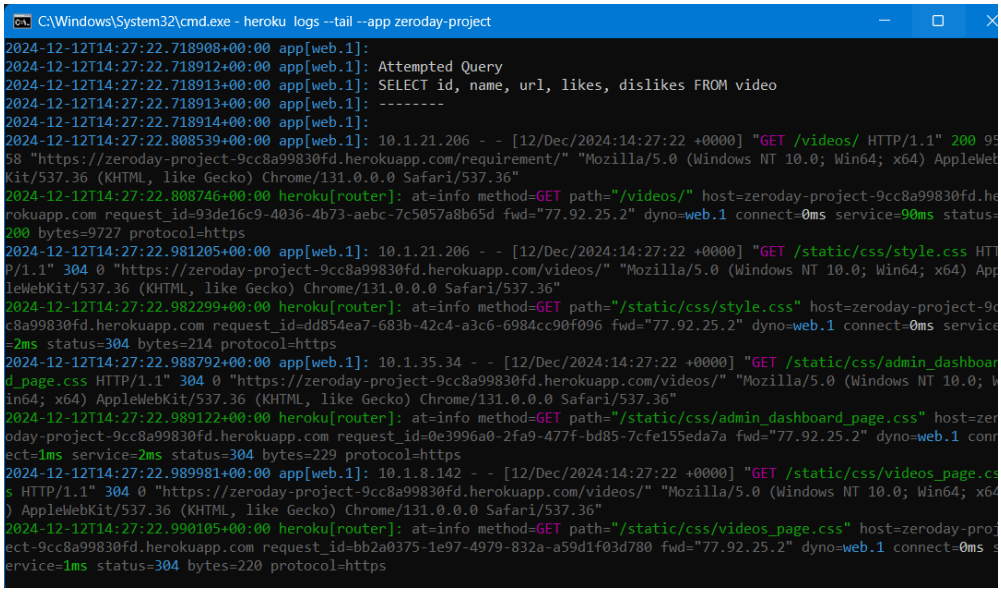


Figure 35: monitoring project

Afterward, I navigated to the directory of my project and reconnected to the database. You can see the connection code in Figure 36. After reconnecting, the issue in my project was resolved.

The problem occurred because the database connection on Heroku was not active or correctly established during the deployment process. The required connection was re-established by reconnecting to the database using the provided Heroku PostgreSQL command, enabling the application to function properly.

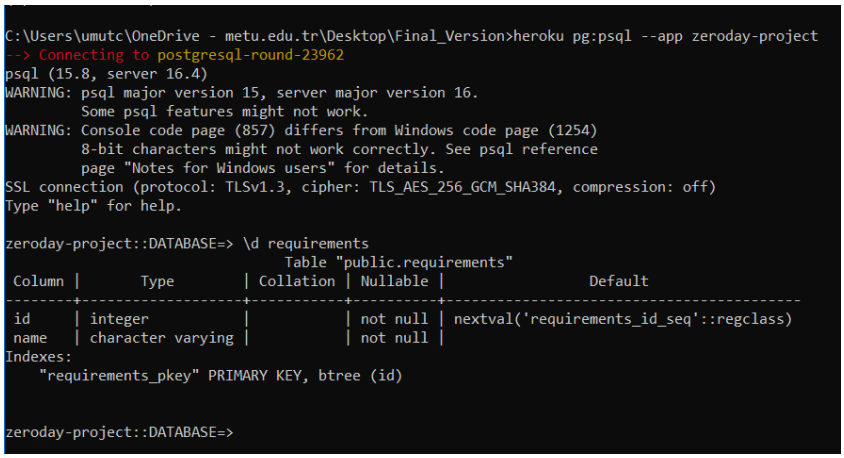


Figure 36: check db connection

I deployed my project to Heroku, allowing me to host my database in the cloud (IaaS) and complete my platform development (PaaS). Since the project is accessible online, I also utilized SaaS services. You can access the details of my project from the links below.

Programming languages and line counts I used in my project:

Python – Flask: 397 lines

HTML: 439 lines

CSS: 171 lines

SQL: 300 lines

You can see in figure 37 what the paid Essential 0 package I use on Heroku Postgres offers me.

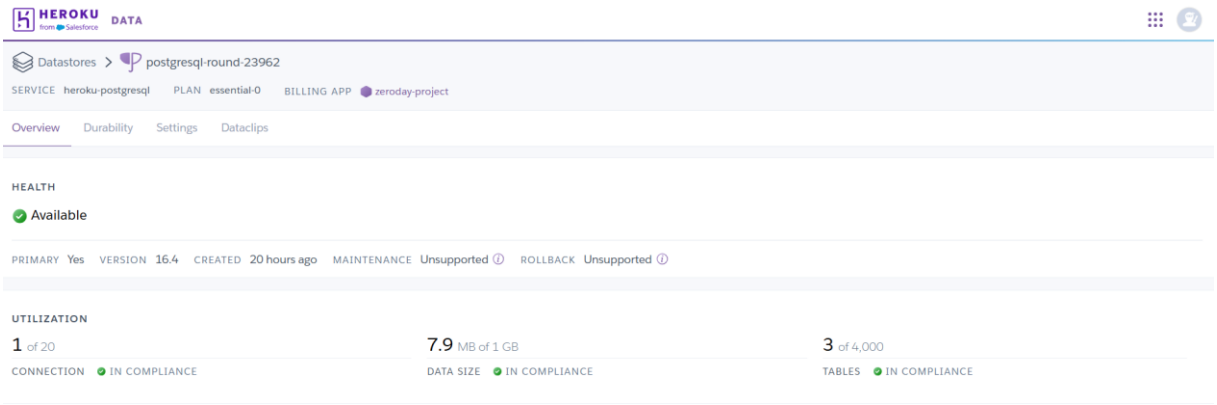


Figure 37: Heroku Postgres utilization

You can find the completed code for now on my github account:

- https://github.com/celikumutcan/ZeroDay-Project
- Github username: celikumutcan

You can also access my working project from the link below:

- https://zeroday-project-9cc8a99830fd.herokuapp.com/

References

1. Platform as a service | Heroku. (n.d.).
   https://www.heroku.com/platform

2. Fully Managed database as a service - PostgreSQL | Heroku. (n.d.).
   https://www.heroku.com/postgres

3. Data security in cloud computing using AES under HEROKU cloud. (2018, April 1). IEEE Conference Publication | IEEE Xplore.
   https://ieeexplore.ieee.org/abstract/document/8372705

4. Andersson, N., & Chernov, A. (2016). Increasing the throughput of a Node.js application : running on the Heroku Cloud App platform. DIVA.

   https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A954978&dswid=5210

5. Booz, R. (2024, September 6). How to Create (Lots of) Sample Time-Series Data With PostgreSQL generate_series(). Timescale Blog.

   https://www.timescale.com/blog/how-to-create-lots-of-sample-time-series-data-with postgresql-generate_series/?utm_source=chatgpt.com

6. SAP SuccessFactors Onboarding | Talent acquisition software. (n.d.). SAP.

   https://www.sap.com/products/hcm/employee-onboarding.html

7. Workday Platform | HR, Finance, Planning, Spend. (n.d.).

   https://www.workday.com/

8. HR Onboarding software for new employees | Create Stellar First Days | BambooHR. (n.d.).

   https://www.bamboohr.com/hr-software/employee-self-onboarding

9. Workspace, G. (n.d.). Google Workspace | Business Apps & Collaboration Tools [Video]. Google Workspace.

   https://workspace.google.com/